WeaveNYC — P2P Architecture Technical Note

Project	WeaveNYC Midterm Proposal (Part 2)
Course	Cloud Computing — Spring 2026
Prepared by	Kairan Shen
Date	March 9, 2026
Purpose	Technical clarification of P2P design, trust boundaries, and failure model

This note provides additional technical detail on the peer-to-peer architecture presented in the midterm proposal. The design separates three planes — communication, governance, and compute — and uses libp2p with GossipSub v1.1 as the community event dissemination layer. The cloud handles search, matching, and notifications but does not own the message plane.

Section 1: Three-Plane Separation

The architecture is cleanest when described as three distinct planes rather than one undifferentiated system.

The communication plane moves community events between participants using a GossipSub v1.1 peer overlay. Edge nodes publish and subscribe directly — no central broker ingests, queues, or redistributes these events. The governance plane decides what an Anchor may approve, hide, tombstone, or escalate. Moderation authority comes from the community role of the Anchor, not from the transport. The compute plane runs managed cloud services — matching, risk detection, geo-search, notifications — none of which participate in the peer overlay.

These three planes interact but carry separate claims. The P2P claim applies to the communication plane. The governance and compute planes are intentionally centralized where appropriate.

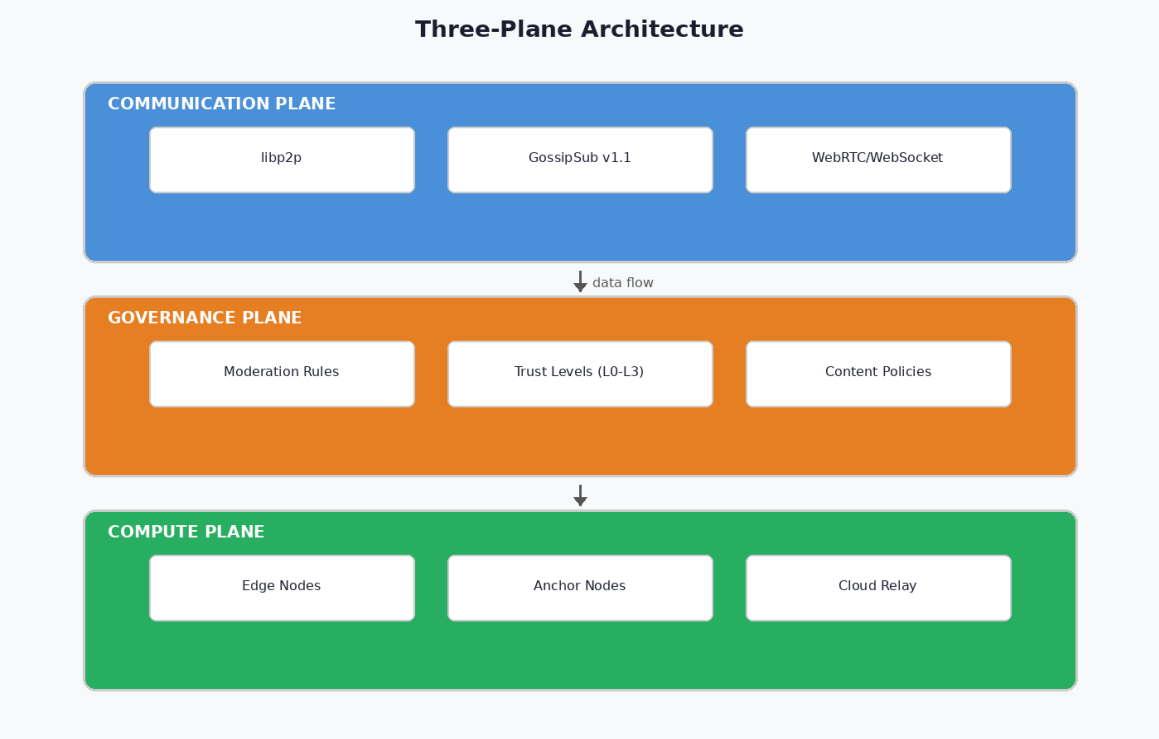


Figure 1: Three-Plane Architecture Overview

Section 2: Component Roles

Component	Responsibility	P2P Status	Detail
Edge Node (PWA)	Creates local drafts, signs events with Ed25519, subscribes to community topics, stores private state in IndexedDB via Dexie.js	Yes — direct peer in the overlay	Max 10 connections, libp2p runs in WebWorker (non-blocking UI thread), WebRTC for direct peer links + WebSocket fallback, DHT disabled on Edge
Anchor Node (Node.js)	Relays community events, verifies signatures, pins shared content on IPFS Helia, applies moderation policy	Peer for transport, authority for governance	Full libp2p node (TCP + WebSocket), DHT enabled, pins all community active posts, hourly cleanup of expired content (>24h)
Cloud Relay	Backup relay when all Anchors are offline, indexes events for search	No — managed service, not a peer	1-2 hardcoded fallback nodes, subscribes to all community topics for backup indexing, TCP transport
Cloud Services (Lambda, Cloud Run, Functions)	Search, matching, risk detection, notifications,	No — compute plane only	Provider-managed PaaS, never touches message

	speech processing		routing

Section 3: GossipSub Topic Structure and Event Model

Each community has three dedicated GossipSub topics:

- weavenyc/{community_id}/posts — post publish, update, delete
- weavenyc/{community_id}/reports — user reports
- weavenyc/{community_id}/announcements — Anchor announcements

Every event follows a typed envelope:

```
type P2PEvent<T> = {  event_id: string;      // UUID v7 (time-ordered, used for dedup)  event_type: string;    // past tense: 'post.published', 'post.updated', etc.  community_id: string;  actor: string;         // derived from Ed25519 public key — NOT user_id  ts: string;            // ISO 8601  payload: T;  signature: string;     // Ed25519 signature };
```

Six event types: post.published, post.updated, post.tombstoned, report.created, moderation.decision, announcement.published. All names use past tense (ADR-0009) to express completed facts, not intentions.

Message size limit: 64 KB per GossipSub message. Post title limited to 60 characters, body to 280 characters.

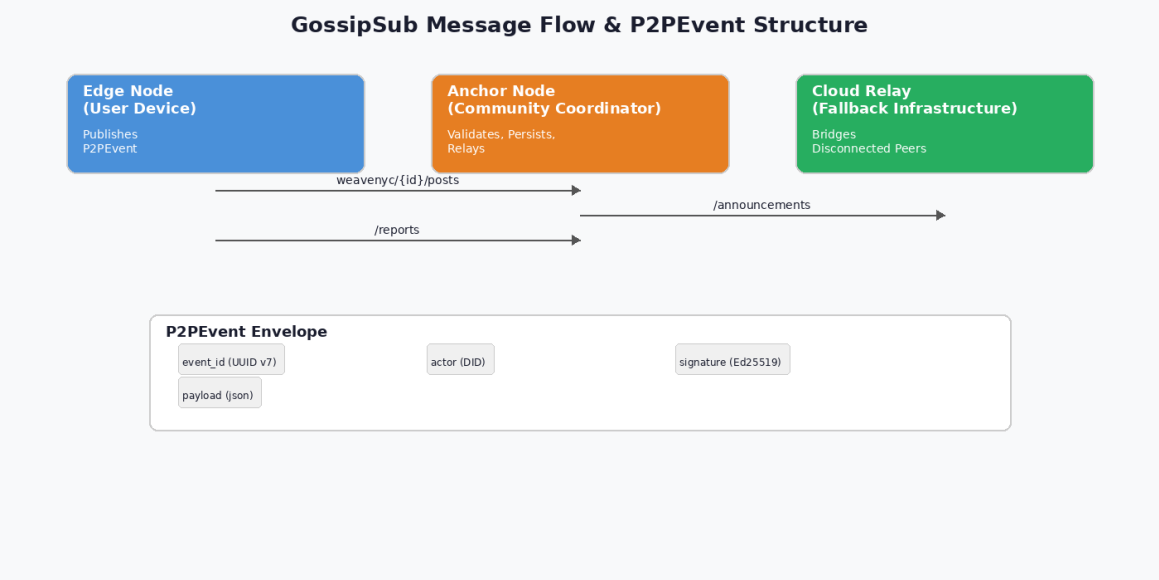


Figure 2: GossipSub Message Flow and P2PEvent Envelope

Section 4: End-to-End Message Flow

Walk through one post from creation to dissemination:

1. User creates a request or offer on the device. The draft stays in IndexedDB (local-only, never synced).

2. On publish, the application packages the event, signs it with the user's Ed25519 private key (stored locally in IndexedDB, never transmitted), and derives an actor ID from the public key. The actor ID is community-scoped — the same user can have different actor IDs across communities to prevent cross-community tracking.

3. The peer publishes the signed event onto weavenyc/{community_id}/posts through the GossipSub overlay. Neighboring peers and/or Anchors relay it onward. WebRTC direct paths are preferred; WebSocket relay paths are used when NAT or browser constraints require them.

4. The Anchor verifies the Ed25519 signature, applies community rules (risk level check, moderation queue for high-risk categories like Childcare and Transportation), and can emit a separate moderation.decision event. That governance event is itself distributed to peers, but its authority comes from the Anchor's community role.

5. Optionally, a minimal anonymized index is written to the cloud (PostgreSQL + PostGIS) for search and matching. This is a secondary step — it is not the primary event transport.

6. Content is stored as a PostSummaryDAG on IPFS Helia — content-addressed, immutable. Updates produce a new CID with a prev_cid pointer forming a version chain.

Section 5: Why GossipSub Is Not Kafka

Question	Peer overlay (libp2p/GossipSub)	Broker system (Kafka/Pub-Sub)
Who receives the initial publish?	Peer publishes to a topic mesh — neighbors relay directly	Client publishes to a central broker cluster
Are direct peer links part of the design?	Yes — Edge-to-Edge via WebRTC, Edge-to-Anchor via WebSocket	No — clients talk to the broker, not to each other
Mandatory center for redistribution?	None — events propagate through the peer mesh	Broker layer is the mandatory redistribution point
What fails if the center is unavailable?	Bootstrap and moderation consistency weaken, but existing peer paths continue disseminating ordinary events	Publication and subscription are blocked or severely degraded
Main abstraction	Peer mesh + topic dissemination	Queue, log, or broker-managed delivery

The claim is narrow: the community event plane uses a genuine peer overlay for dissemination, not a set of clients publishing into one mandatory broker.

Section 6: Bootstrap, Signaling, and Relay

A browser-based P2P system cannot avoid coordination at the network edge. Bootstrap, signaling, and relay are standard support mechanisms in libp2p — they do not erase the P2P character of the event plane.

Bootstrap process for a new Edge node:

1. Load known Anchor addresses from local cache (5 recently successful addresses) plus hardcoded fallback list.

2. Connect via WebSocket to the Anchor's REST endpoint: GET /v1/communities/{id}/bootstrap.

3. Receive a snapshot: recent posts + peer list, all Ed25519 verified.

4. Subscribe to 3 GossipSub topics (posts, reports, announcements) and begin incremental sync from the overlay backlog.

If all Anchors are unreachable, 1-2 hardcoded Cloud relay nodes serve as final-fallback bootstrap peers. This is the same pattern Bitcoin uses with DNS seeds and IPFS uses with bootstrap nodes.

WebRTC handles direct browser-to-browser data exchange. Signaling (through the Anchor or Cloud relay) helps peers discover each other and negotiate connectivity. Relay assists when two peers cannot form a direct NAT-traversed path. None of these are the same function as a broker that owns message redistribution.

Section 7: Anchor Failover

Anchors use a fixed hierarchy rather than dynamic election:

Role	Sync Interval	Behavior
Primary Anchor	—	Handles all governance events, syncs state to secondaries
Secondary #1 (Hot Standby)	Receives state every 30 seconds	Auto-promotes to Primary if Primary is unreachable
Secondary #2 (Cold Standby)	Receives state every 60 seconds	Promotes if both Primary and #1 are down

Why fixed hierarchy, not Raft or Paxos?

Three reasons. First, no split-brain risk — a fixed hierarchy avoids the complexity of consensus protocols. Second, community trust — the Anchor is a known organization (church, NGO), not a random node winning an election. Third, Edge-to-Edge messaging continues without any Anchor — peers lose governance consistency but retain communication availability.

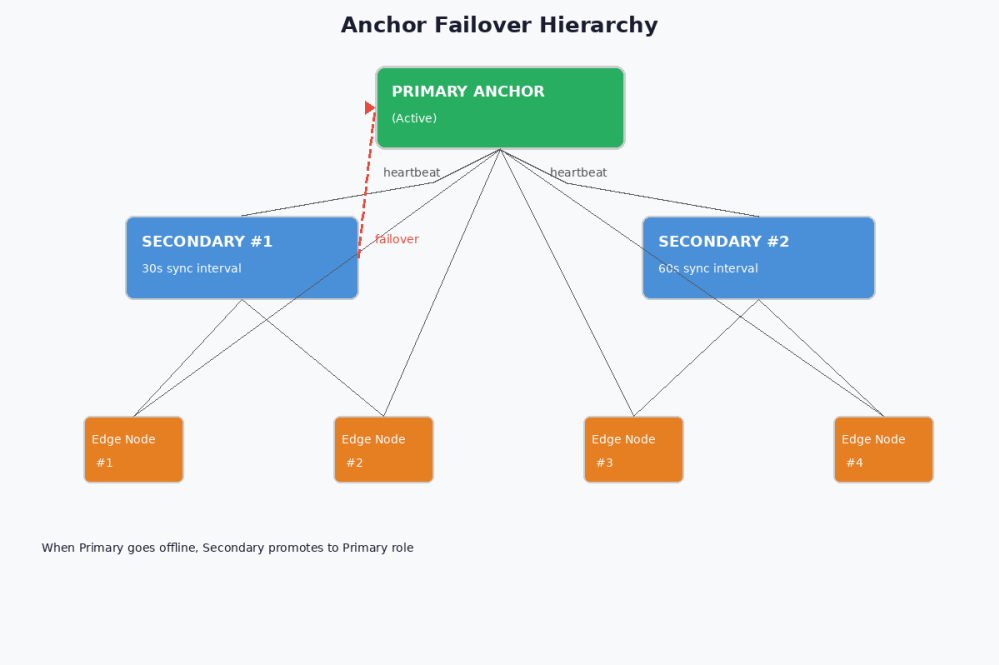


Figure 3: Anchor Failover Hierarchy

Section 8: What Happens When an Anchor Is Unavailable

Capability	Status When Anchor Offline
Ordinary peer-to-peer dissemination	Continues among already-connected peers
New-node bootstrap	Harder — falls back to cached addresses or Cloud relay
Moderation consistency	Weaker — governance authority temporarily absent
Content pinning / shared retention	Degrades if the Anchor was responsible for IPFS pinning
Low-risk post display	Still available from IndexedDB cache
High-risk post moderation	Stalled until an Anchor returns

The architecture is a governance-layered hybrid: communication availability and governance availability are related but not identical.

Section 9: Consistency Model

The proposal uses eventual consistency (CAP: AP) with application-level conflict rules, not strong consensus.

Mechanism	Detail
Event dedup	event_id (UUID v7) — duplicate gossip is dropped

Version chain	Each post update produces a new IPFS CID with prev_cid pointer — immutable audit trail
Post conflicts	Last-write-wins by incrementing version number
Moderation conflicts	Anchor-wins — governance events override ordinary content state
Tombstone	post.tombstoned event broadcasts deletion, all nodes unpin the CID
Expiry cleanup	Anchor runs hourly task: unpin posts expired > 24 hours

This is a practical, bounded consistency model appropriate for semester scope. It is not a claim of full distributed agreement or blockchain-grade consensus.

Section 10: Summary

The design uses a genuine peer overlay (libp2p + GossipSub v1.1) for community event dissemination, not a cloud message bus with a decentralized label. Peers publish, relay, and receive events directly through the overlay. The Anchor adds governance authority on top of this transport — it is a peer for communication and a trusted role for moderation. The cloud provides compute, search, and notifications but does not own the message plane. Bootstrap, signaling, and relay are standard P2P support mechanisms that coexist with the peer overlay rather than replacing it.

References:

[1] libp2p documentation — https://libp2p.io/docs/

[2] libp2p Publish/Subscribe — https://libp2p.io/guides/pubsub/

[3] W3C WebRTC specification — https://www.w3.org/TR/webrtc/

[4] IPFS / Helia documentation — https://helia.io/

[5] IETF RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA)